Slip 1

Q1) Write a program to implement Breadth First Search (BFS) for the Monkey Banana Problem. [10 marks]

```
from collections import deque

def monkey_banana_bfs():
    start_state = ('door', 'window', False, False)
    goal_state = ('middle', 'middle', True, True)
    positions = ['door', 'window', 'middle']

    queue = deque([(start_state, [])])
    visited = {start_state}

    while queue:
        state, path = queue.popleft()
        m_pos, b_pos, on_box, has_banana = state

        if state == goal_state:
            return path

        next_moves = []

        if not on_box:
            for p in positions:
                if p != m_pos:
                    next_moves.append((
                        (p, b_pos, False, has_banana),
                        f"Walk from {m_pos} to {p}"
                    ))
                if m_pos == b_pos and p != b_pos:
                    next_moves.append((
                        (p, p, False, has_banana),
                        f"Push box from {b_pos} to {p}"
                    ))
            if m_pos == b_pos:
                next_moves.append((
                    (m_pos, b_pos, True, has_banana),
                    "Climb onto the box"
                ))
        elif on_box and m_pos == 'middle' and not has_banana:
            next_moves.append((
                (m_pos, b_pos, True, True),
                "Grasp the banana"
            ))

        for next_state, action in next_moves:
            if next_state not in visited:
                visited.add(next_state)
                queue.append((next_state, path + [action]))

    return None

steps = monkey_banana_bfs()
```

```
for i, step in enumerate(steps, 1):
    print(f"Step {i}: {step}")
```

Q2) Write a program to implement the A* Search Algorithm to find the shortest path between a source node and a destination node. [20 marks]

```
import heapq

graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('D', 2), ('E', 5)],
    'C': [('F', 3)],
    'D': [('G', 3)],
    'E': [('G', 1)],
    'F': [('G', 2)],
    'G': []
}

heuristics = {
    'A': 6, 'B': 5, 'C': 4, 'D': 3, 'E': 2, 'F': 2, 'G': 0
}

def a_star_search(graph, h, start, goal):
    open_list = [(h[start], 0, start, [start])]
    visited = {}

    while open_list:
        f, g, current, path = heapq.heappop(open_list)

        if current == goal:
            return path, g

        if current in visited and visited[current] <= g:
            continue
        visited[current] = g

        for neighbor, weight in graph[current]:
            new_g = g + weight
            new_f = new_g + h[neighbor]
            next_path = path + [neighbor]
            heapq.heappush(open_list, (new_f, new_g, neighbor, next_path))

    return None, float('inf')

path, cost = a_star_search(graph, heuristics, 'A', 'G')
print("Shortest Path:", " -> ".join(path))
print("Total Minimum Cost:", cost)
```

OR Q2) Write a program to solve the Map Coloring Problem using the Constraint Satisfaction Problem (CSP) approach. [20 marks]

```
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D'],
    'C': ['A', 'D'],
    'D': ['B', 'C']
```

```
}

colors = ['Red', 'Green', 'Blue']

def is_safe(node, color, assignment, graph):
    for neighbor in graph[node]:
        if assignment.get(neighbor) == color:
            return False
    return True

def solve_map_coloring(nodes, graph, colors, assignment={}):
    if len(assignment) == len(nodes):
        return assignment

    unassigned = [n for n in nodes if n not in assignment][0]

    for color in colors:
        if is_safe(unassigned, color, assignment, graph):
            assignment[unassigned] = color
            result = solve_map_coloring(nodes, graph, colors, assignment)
            if result:
                return result
            del assignment[unassigned]

    return None

nodes = list(graph.keys())
solution = solve_map_coloring(nodes, graph, colors, {})
for node, color in solution.items():
    print(f"Node {node} -> {color}")
```

Slip 2

Q1) Write a program to solve the Water Jug Problem using BFS. [10 marks]

```
from collections import deque

def water_jug_bfs(jug1_cap, jug2_cap, target):
    start = (0, 0)
    queue = deque([(start, [start])])
    visited = {start}

    while queue:
        (x, y), path = queue.popleft()

        if x == target or y == target:
            return path

        pour1to2 = min(x, jug2_cap - y)
        pour2to1 = min(y, jug1_cap - x)

        next_states = [
            (jug1_cap, y),
            (x, jug2_cap),
```

```
            (0, y),
            (x, 0),
            (x - pour1to2, y + pour1to2),
            (x + pour2to1, y - pour2to1)
        ]

        for state in next_states:
            if state not in visited:
                visited.add(state)
                queue.append((state, path + [state]))

    return None

path = water_jug_bfs(4, 3, 2)
for step in path:
    print(step)
```

Q2) Write a program to implement the Backtracking Algorithm for solving a constraint satisfaction problem. [20 marks]

```
graph = {
    1: [2, 3, 4, 5],
    2: [1, 3, 4, 5],
    3: [1, 2, 4, 5],
    4: [1, 2, 3, 5],
    5: [1, 2, 3, 4]
}

colors = ['Red', 'Green', 'Blue', 'Yellow', 'Purple']

def is_valid(node, color, assignment, graph):
    for neighbor in graph[node]:
        if assignment.get(neighbor) == color:
            return False
    return True

def backtrack_csp(nodes, graph, colors, assignment={}):
    if len(assignment) == len(nodes):
        return assignment

    node = [n for n in nodes if n not in assignment][0]

    for color in colors:
        if is_valid(node, color, assignment, graph):
            assignment[node] = color
            result = backtrack_csp(nodes, graph, colors, assignment)
            if result:
                return result
            del assignment[node]

    return None

nodes = list(graph.keys())
solution = backtrack_csp(nodes, graph, colors, {})
```

```
for node, color in solution.items():
    print(f"Vertex {node}: {color}")
```

OR Q2) Write a program to implement a Game Tree Representation for a two-player game. [20 marks]

```
game_tree = {
    'Root': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': 3,
    'E': 5,
    'F': 2,
    'G': 9
}

def display_tree(node, level=0):
    indent = "    " * level
    if isinstance(game_tree[node], int):
        print(f"{indent}Leaf {node}: Value = {game_tree[node]}")
    else:
        player = "MAX" if level % 2 == 0 else "MIN"
        print(f"{indent}Node {node} ({player}):")
        for child in game_tree[node]:
            display_tree(child, level + 1)

def evaluate_game_tree(node, is_max):
    if isinstance(game_tree[node], int):
        return game_tree[node]

    children = game_tree[node]
    vals = [evaluate_game_tree(c, not is_max) for c in children]
    return max(vals) if is_max else min(vals)

display_tree('Root')
print("Optimal Score at Root:", evaluate_game_tree('Root', True))
```

Slip 3

Q1) Write a program to implement BFS traversal for a graph. [10 marks]

```
from collections import deque

graph = {
    1: [2, 3],
    2: [4, 5],
    3: [6, 7],
    4: [8, 9],
    5: [10, 11],
    6: [12, 13],
    7: [14, 15],
    8: [], 9: [], 10: [], 11: [], 12: [], 13: [], 14: [], 15: []
}

def bfs_traversal(graph, start_node):
```

```
    visited = set([start_node])
    queue = deque([start_node])
    order = []

    while queue:
        node = queue.popleft()
        order.append(node)

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

    return order

traversal = bfs_traversal(graph, 1)
print("BFS Traversal Order:", " -> ".join(map(str, traversal)))
```

Q2) Write a program to implement Alpha-Beta Pruning for optimizing the Minimax search process. [20 marks]

```
import math

tree = {
    'Root': ['A', 'B', 'C'],
    'A': [3, 5],
    'B': [2, 9],
    'C': [0, 7]
}

def alpha_beta(node, is_max, alpha, beta):
    if isinstance(node, int):
        return node

    if is_max:
        best = -math.inf
        for child in tree[node]:
            val = alpha_beta(child, False, alpha, beta)
            best = max(best, val)
            alpha = max(alpha, best)
            if beta <= alpha:
                print(f"Pruned at MAX node {node}")
                break
        return best
    else:
        best = math.inf
        for child in tree[node]:
            val = alpha_beta(child, True, alpha, beta)
            best = min(best, val)
            beta = min(beta, best)
            if beta <= alpha:
                print(f"Pruned after {child} at MIN node {node}")
                break
        return best
```

```
optimal_value = alpha_beta('Root', True, -math.inf, math.inf)
print("Optimal Value using Alpha-Beta Pruning:", optimal_value)
```

OR Q2) Write a program to implement Propositional Logic and evaluate logical expressions using operators such as AND, OR, and NOT. [20 marks]

```
def evaluate_propositional_logic():
    print("P    Q       NOT P   P AND Q P OR Q  (P AND Q) OR (NOT P)")
    print("-" * 60)

    for P in [True, False]:
        for Q in [True, False]:
            not_p = not P
            p_and_q = P and Q
            p_or_q = P or Q
            expr = (P and Q) or (not P)
            print(f"{P} {Q}     {not_p} {p_and_q}       {p_or_q}        {expr}")

evaluate_propositional_logic()
```

Slip 4

Q1) Write a program to solve a state-space search problem using BFS. Start Vertex = A [10 marks]

```
from collections import deque

graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D'],
    'C': ['A', 'E'],
    'D': ['B', 'E'],
    'E': ['C', 'D']
}

def bfs_state_space(graph, start):
    visited = set([start])
    queue = deque([start])
    order = []

    while queue:
        vertex = queue.popleft()
        order.append(vertex)

        for neighbor in graph[vertex]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

    return order

result = bfs_state_space(graph, 'A')
print("BFS Traversal from Start Vertex A:", " -> ".join(result))
```

Q2) Write a program to implement Predicate Logic using predicates, variables, and quantifier. [20 marks]

```
numbers = [2, 4, 6, 8, 10]
people = ['Socrates', 'Plato', 'Aristotle']
mortals = {'Socrates', 'Plato', 'Aristotle'}

def is_even(x):
    return x % 2 == 0

def is_greater_than_8(x):
    return x > 8

def is_mortal(person):
    return person in mortals

all_even = all(is_even(x) for x in numbers)
all_mortal = all(is_mortal(p) for p in people)
exists_gt_8 = any(is_greater_than_8(x) for x in numbers)

print("Domain of Numbers:", numbers)
print("Universal (For All is_even):", all_even)
print("Existential (Exists is_greater_than_8):", exists_gt_8)
print("Universal (For All is_mortal):", all_mortal)
```

OR Q2) Write a program to implement Gaussian NB classifier with 3 classes on numeric data, use make_blobs to create a dataset X of 200 sample points in 2D Space, 3 clusters and each cluster spread (SD) over 1.5. [20 marks]

```
from sklearn.datasets import make_blobs
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

X, y = make_blobs(
    n_samples=200,
    n_features=2,
    centers=[[-2, 5], [0, 0], [6, -0.3]],
    cluster_std=1.5,
    random_state=42
)

model = GaussianNB()
model.fit(X, y)

X_test = [[-2, 5], [0, 0], [6, -0.3]]
predictions = model.predict(X_test)

y_pred_all = model.predict(X)
acc = accuracy_score(y, y_pred_all)

print("Input (Values):", X_test)
print("Output:", predictions)
print("Accuracy Score:", round(acc, 4))
```

Slip 7

Q1) Write a program to implement the A* Search Algorithm to find the shortest path between a source node and a destination node (Source = V1, Destination = V6). [10 marks]

```
import heapq

graph = {
    'V1': [('V2', 2), ('V3', 4)],
    'V2': [('V4', 3)],
    'V3': [('V4', 1), ('V5', 5)],
    'V4': [('V6', 2)],
    'V5': [('V6', 1)],
    'V6': []
}

heuristics = {
    'V1': 6, 'V2': 4, 'V3': 3, 'V4': 2, 'V5': 1, 'V6': 0
}

def a_star(graph, h, start, goal):
    pq = [(h[start], 0, start, [start])]
    visited = {}

    while pq:
        f, g, node, path = heapq.heappop(pq)

        if node == goal:
            return path, g

        if node in visited and visited[node] <= g:
            continue
        visited[node] = g

        for neighbor, weight in graph[node]:
            new_g = g + weight
            new_f = new_g + h[neighbor]
            heapq.heappush(pq, (new_f, new_g, neighbor, path + [neighbor]))

    return None, float('inf')

path, cost = a_star(graph, heuristics, 'V1', 'V6')
print("Shortest Path from V1 to V6:", " -> ".join(path))
print("Total Path Cost:", cost)
```

Q2) Write a program to solve the Map Coloring Problem using the Constraint Satisfaction Problem (CSP) approach. [20 marks]

```
graph = {
    'A': ['B', 'E'],
    'B': ['A', 'C'],
    'C': ['B', 'D'],
    'D': ['C', 'E'],
    'E': ['D', 'A']
}
```

```
colors = ['Red', 'Green', 'Blue']

def is_safe(node, color, assignment, graph):
    for neighbor in graph[node]:
        if assignment.get(neighbor) == color:
            return False
    return True

def solve_csp(nodes, graph, colors, assignment={}):
    if len(assignment) == len(nodes):
        return assignment

    unassigned = [n for n in nodes if n not in assignment][0]

    for color in colors:
        if is_safe(unassigned, color, assignment, graph):
            assignment[unassigned] = color
            result = solve_csp(nodes, graph, colors, assignment)
            if result:
                return result
            del assignment[unassigned]

    return None

solution = solve_csp(list(graph.keys()), graph, colors, {})
for node, color in solution.items():
    print(f"Node {node} -> {color}")
```

OR Q2) Write a program to develop a Tic-Tac-Toe Game in which the computer uses the Minimax Algorithm to play optimally. [20 marks]

```
import math

board = [' ' for _ in range(9)]

def print_board(b):
    for i in range(0, 9, 3):
        print(f" {b[i]} | {b[i+1]} | {b[i+2]} ")
        if i < 6:
            print("---+---+---")

def check_winner(b):
    win_lines = [
        (0, 1, 2), (3, 4, 5), (6, 7, 8),
        (0, 3, 6), (1, 4, 7), (2, 5, 8),
        (0, 4, 8), (2, 4, 6)
    ]
    for x, y, z in win_lines:
        if b[x] == b[y] == b[z] and b[x] != ' ':
            return b[x]
    if ' ' not in b:
        return 'Draw'
    return None

def minimax(b, is_max):
```

```
    winner = check_winner(b)
    if winner == 'O':
        return 1
    if winner == 'X':
        return -1
    if winner == 'Draw':
        return 0

    if is_max:
        best = -math.inf
        for i in range(9):
            if b[i] == ' ':
                b[i] = 'O'
                best = max(best, minimax(b, False))
                b[i] = ' '
        return best
    else:
        best = math.inf
        for i in range(9):
            if b[i] == ' ':
                b[i] = 'X'
                best = min(best, minimax(b, True))
                b[i] = ' '
        return best

def best_computer_move(b):
    best_val = -math.inf
    move = -1
    for i in range(9):
        if b[i] == ' ':
            b[i] = 'O'
            move_val = minimax(b, False)
            b[i] = ' '
            if move_val > best_val:
                best_val = move_val
                move = i
    return move

human_moves = [0, 8, 6, 3]
for h_move in human_moves:
    if board[h_move] == ' ' and not check_winner(board):
        board[h_move] = 'X'
        if not check_winner(board):
            c_move = best_computer_move(board)
            if c_move != -1:
                board[c_move] = 'O'

print_board(board)
print("Result:", check_winner(board))
```

Slip 8

Q1) Write a program to solve the 4-Puzzle Problem using the A* Search Algorithm. [10 marks]

```
import heapq

start_state = (2, 0, 1, 3)
goal_state = (1, 2, 3, 0)

moves = {
    0: [1, 2],
    1: [0, 3],
    2: [0, 3],
    3: [1, 2]
}

def heuristic(state, goal):
    return sum(1 for i in range(4) if state[i] != 0 and state[i] != goal[i])

def a_star_4_puzzle(start, goal):
    pq = [(heuristic(start, goal), 0, start, [start])]
    visited = {start: 0}

    while pq:
        f, g, current, path = heapq.heappop(pq)

        if current == goal:
            return path

        zero_idx = current.index(0)
        for next_idx in moves[zero_idx]:
            b = list(current)
            b[zero_idx], b[next_idx] = b[next_idx], b[zero_idx]
            next_state = tuple(b)

            if next_state not in visited or g + 1 < visited[next_state]:
                visited[next_state] = g + 1
                new_f = g + 1 + heuristic(next_state, goal)
                next_path = path + [next_state]
                heapq.heappush(pq, (new_f, g + 1, next_state, next_path))

    return None

solution = a_star_4_puzzle(start_state, goal_state)
for step_num, state in enumerate(solution):
    print(f"Step {step_num}: {state[:2]} / {state[2:]}")
```

Q2) Write a program or report that compares Minimax and Alpha-Beta Pruning on the same game tree. [20 marks]

```
import math

leaves = [3, 5, 6, 9, 1, 2, 0, -1]
minimax_nodes = 0
alphabeta_nodes = 0
```

```
def minimax(depth, idx, is_max):
    global minimax_nodes
    minimax_nodes += 1
    if depth == 3:
        return leaves[idx]

    if is_max:
        return max(minimax(depth + 1, idx * 2, False),
                   minimax(depth + 1, idx * 2 + 1, False))
    else:
        return min(minimax(depth + 1, idx * 2, True),
                   minimax(depth + 1, idx * 2 + 1, True))

def alpha_beta(depth, idx, is_max, alpha, beta):
    global alphabeta_nodes
    alphabeta_nodes += 1
    if depth == 3:
        return leaves[idx]

    if is_max:
        best = -math.inf
        for i in range(2):
            val = alpha_beta(depth + 1, idx * 2 + i, False, alpha, beta)
            best = max(best, val)
            alpha = max(alpha, best)
            if beta <= alpha:
                break
        return best
    else:
        best = math.inf
        for i in range(2):
            val = alpha_beta(depth + 1, idx * 2 + i, True, alpha, beta)
            best = min(best, val)
            beta = min(beta, best)
            if beta <= alpha:
                break
        return best

mm_val = minimax(0, 0, True)
ab_val = alpha_beta(0, 0, True, -math.inf, math.inf)

print(f"Minimax Value    : {mm_val} | Nodes Visited: {minimax_nodes}")
print(f"Alpha-Beta Value : {ab_val} | Nodes Visited: {alphabeta_nodes}")
print(f"Nodes Saved      : {minimax_nodes - alphabeta_nodes}")
```

OR Q2) Write a program to design a Rule-Based Expert System for simple decision-making. [20 marks]

```
def medical_expert_system(symptoms):
    rules = [
        ({"fever", "cough", "loss_of_taste"}, "COVID-19"),
        ({"fever", "cough", "body_ache"}, "Flu (Influenza)"),
        ({"sneezing", "runny_nose"}, "Common Cold"),
        ({"headache", "nausea", "sensitivity_to_light"}, "Migraine")
    ]
```

```
    for required_symptoms, diagnosis in rules:
        if required_symptoms.issubset(symptoms):
            return f"Diagnosis: {diagnosis}"

    return "Diagnosis: General viral infection."

patient_1 = {"fever", "cough", "body_ache"}
patient_2 = {"sneezing", "runny_nose"}

print("Patient 1:", medical_expert_system(patient_1))
print("Patient 2:", medical_expert_system(patient_2))
```

Slip 9

Q1) Write a program to demonstrate the Hill Climbing Algorithm for finding the maximum value of a simple objective function. [10 marks]

```
def objective_function(x):
    return -((x - 3) ** 2) + 9

def hill_climbing(start_x, step_size=0.5):
    current_x = start_x
    current_val = objective_function(current_x)

    while True:
        neighbors = [current_x - step_size, current_x + step_size]
        best_x = max(neighbors, key=objective_function)
        best_val = objective_function(best_x)

        if best_val <= current_val:
            break

        current_x, current_val = best_x, best_val

    return current_x, current_val

max_x, max_val = hill_climbing(start_x=0.0)
print(f"Maximum found at x = {max_x}")
print(f"Maximum Value f(x) = {max_val}")
```

Q2) Write a program to implement the Forward Chaining inference mechanism. [20 marks]

```
def forward_chaining(facts, rules, goal):
    inferred = set(facts)
    changed = True

    while changed:
        changed = False
        for premises, conclusion in rules:
            if premises.issubset(inferred) and (conclusion not in inferred):
                inferred.add(conclusion)
                print(f"Rule fired: {premises} -> {conclusion}")
                changed = True
                if conclusion == goal:
```

```
                    return True, inferred

    return (goal in inferred), inferred

facts = {"has_feathers", "flies", "lays_eggs"}
rules = [
    ({"has_feathers"}, "is_bird"),
    ({"is_bird", "flies"}, "can_migrate"),
    ({"can_migrate", "lays_eggs"}, "is_wild_bird")
]
goal = "is_wild_bird"

reached, all_facts = forward_chaining(facts, rules, goal)
print(f"Goal '{goal}' Reached:", reached)
```

OR Q2) Write a program to implement the Decision Tree Classifier for a classification problem. [20 marks]

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

data = {
    'Pregnancies': [6, 1, 8, 1, 0, 5, 3, 10, 2, 8, 4],
    'Glucose': [148, 85, 183, 89, 137, 116, 78, 115, 197, 125, 110],
    'BP': [72, 66, 64, 66, 40, 74, 50, 0, 70, 96, 92],
    'SkinThick': [35, 29, 0, 23, 35, 0, 32, 0, 45, 0, 0],
    'Insulin': [0, 0, 0, 94, 168, 0, 88, 0, 543, 0, 0],
    'BMI': [33.6, 26.6, 23.3, 28.1, 43.1, 25.6,
            31.0, 35.3, 30.5, 0.0, 37.6],
    'DiabetesPF': [0.627, 0.351, 0.672, 0.167, 2.288, 0.201,
                   0.248, 0.134, 0.158, 0.232, 0.191],
    'Age': [50, 31, 32, 21, 33, 30, 26, 29, 53, 54, 30],
    'Outcome': [1, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0]
}

df = pd.DataFrame(data)
X = df.drop('Outcome', axis=1)
y = df['Outcome']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42
)

dt_gini = DecisionTreeClassifier(random_state=42)
dt_gini.fit(X_train, y_train)
gini_acc = accuracy_score(y_test, dt_gini.predict(X_test))
print("Default (Gini) Accuracy:", round(gini_acc, 4))

for depth in [2, 3, 4]:
    dt_ent = DecisionTreeClassifier(
        criterion='entropy', max_depth=depth, random_state=42
    )
    dt_ent.fit(X_train, y_train)
```

```
    acc = accuracy_score(y_test, dt_ent.predict(X_test))
    print(f"Entropy (max_depth={depth}) Accuracy: {round(acc, 4)}")
```

Slip 14

Q1) Write a program to implement BFS traversal for a graph. [10 marks]

```
from collections import deque

graph = {
    'V1': ['V2', 'V5', 'V3'],
    'V2': ['V1', 'V4'],
    'V3': ['V1', 'V6'],
    'V4': ['V2', 'V5'],
    'V5': ['V1', 'V4', 'V6'],
    'V6': ['V3', 'V5']
}

def bfs_traversal(graph, start):
    visited = set([start])
    queue = deque([start])
    order = []

    while queue:
        node = queue.popleft()
        order.append(node)

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

    return order

print("BFS Traversal:", " -> ".join(bfs_traversal(graph, 'V1')))
```

Q2) Write a program or report to compare Supervised Learning and Unsupervised Learning techniques. [20 marks]

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.cluster import KMeans
from sklearn.metrics import accuracy_score, silhouette_score

iris = load_iris()
X, y = iris.data, iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
clf = DecisionTreeClassifier(random_state=42)
clf.fit(X_train, y_train)
sup_acc = accuracy_score(y_test, clf.predict(X_test))
```

```
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
clusters = kmeans.fit_predict(X)
unsup_sil = silhouette_score(X, clusters)

print(f"Supervised (Decision Tree) Accuracy : {sup_acc:.4f}")
print(f"Unsupervised (K-Means) Silhouette   : {unsup_sil:.4f}")
```

OR Q2) Write a program to demonstrate basic Ontological Engineering concepts using classes, subclasses and relationships. [20 marks]

```
class Entity:
    def __init__(self, name):
        self.name = name

class Animal(Entity):
    category = "Living Being"

class Mammal(Animal):
    has_Trait = "Warm-blooded"

class Dog(Mammal):
    sound = "Barks"

buddy = Dog("Buddy")

print("Instance Name:", buddy.name)
print("Is Dog a subclass of Mammal?:", issubclass(Dog, Mammal))
print("Is Dog a subclass of Animal?:", issubclass(Dog, Animal))
print("Inherited Category:", buddy.category)
print("Inherited Trait:", buddy.has_Trait)
print("Specific Property:", buddy.sound)
```

Slip 15

Q1) Write a program to solve a state-space search problem using DFS. Start vertex - 1 [10 marks]

```
graph = {
    1: [2, 3],
    2: [1, 4],
    3: [1, 5],
    4: [2, 5, 6],
    5: [3, 4, 6],
    6: [4, 5]
}

def dfs_traversal(graph, node, visited=None, order=None):
    if visited is None:
        visited = set()
        order = []

    visited.add(node)
    order.append(node)

    for neighbor in graph[node]:
```

```
        if neighbor not in visited:
            dfs_traversal(graph, neighbor, visited, order)

    return order

order = dfs_traversal(graph, 1)
print("DFS Traversal:", " -> ".join(map(str, order)))
```

Q2) Write Python program to demonstrate clustering objective functions and plot across K using matplotlib. [20 marks]

```
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, davies_bouldin_score

X, _ = make_blobs(
    n_samples=250, centers=4, cluster_std=1.0, random_state=42
)

k_values = range(2, 8)
wcss_list = []
silhouette_list = []
db_index_list = []

for k in k_values:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(X)

    wcss = kmeans.inertia_
    sil = silhouette_score(X, labels)
    db = davies_bouldin_score(X, labels)

    wcss_list.append(wcss)
    silhouette_list.append(sil)
    db_index_list.append(db)
    print(f"K={k} | WCSS={wcss:.2f} | Sil={sil:.3f} | DB={db:.3f}")

plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.plot(k_values, wcss_list, marker='o')
plt.title("Elbow Curve (WCSS)")
plt.xlabel("K")
plt.ylabel("WCSS")

plt.subplot(1, 3, 2)
plt.plot(k_values, silhouette_list, marker='s', color='green')
plt.title("Silhouette Score")
plt.xlabel("K")
plt.ylabel("Score")

plt.subplot(1, 3, 3)
plt.plot(k_values, db_index_list, marker='^', color='red')
plt.title("Davies-Bouldin Index")
```

```
plt.xlabel("K")
plt.ylabel("DB Score")

plt.tight_layout()
plt.show()
```

OR Q2) Write a program to build a Knowledge Base using logical rules and facts. [20 marks]

```
parents = [
    ("Raj", "Amit"),
    ("Raj", "Neha"),
    ("Amit", "Rohan"),
    ("Neha", "Priya")
]

def is_sibling(x, y):
    if x == y:
        return False
    for p1, c1 in parents:
        for p2, c2 in parents:
            if p1 == p2 and c1 == x and c2 == y:
                return True
    return False

def is_grandparent(x, z):
    for p1, y in parents:
        if p1 == x:
            for p2, c2 in parents:
                if p2 == y and c2 == z:
                    return True
    return False

print("Is Amit a sibling of Neha?     :", is_sibling("Amit", "Neha"))
print("Is Raj a grandparent of Rohan? :", is_grandparent("Raj", "Rohan"))
print("Is Raj a grandparent of Amit?  :", is_grandparent("Raj", "Amit"))
```

Slip 16

Q1) Write a program to implement Depth Limited Search (DLS). [10 marks]

```
graph = {
    1: [2, 3],
    2: [4, 5],
    3: [6],
    4: [],
    5: [],
    6: []
}

def depth_limited_search(graph, current, goal, limit, path=None):
    if path is None:
        path = [current]

    if current == goal:
        return path
```

```
    if limit <= 0:
        return None

    for neighbor in graph[current]:
        next_path = path + [neighbor]
        result = depth_limited_search(
            graph, neighbor, goal, limit - 1, next_path
        )
        if result is not None:
            return result

    return None

path = depth_limited_search(graph, 1, 5, 2)
print("DLS Path:", " -> ".join(map(str, path)))
```

Q2) Write a program to implement the Backward Chaining inference mechanism. [20 marks]

```
facts = {"has_fur", "eats_meat", "has_tawny_color", "has_dark_spots"}

rules = {
    "is_mammal": ["has_fur"],
    "is_carnivore": ["is_mammal", "eats_meat"],
    "is_cheetah": ["is_carnivore", "has_tawny_color", "has_dark_spots"]
}

def backward_chaining(goal):
    if goal in facts:
        print(f"Fact verified: '{goal}'")
        return True

    if goal in rules:
        print(f"Proving sub-goals for '{goal}': {rules[goal]}")
        return all(backward_chaining(sub) for sub in rules[goal])

    return False

goal = "is_cheetah"
result = backward_chaining(goal)
print(f"Goal '{goal}' proved:", result)
```

OR Q2) Write a program to implement the K-Means Clustering Algorithm on country_data.csv. [20 marks]

```
import os
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

if os.path.exists("country_data.csv"):
    df = pd.read_csv("country_data.csv")
else:
    df = pd.DataFrame({
        'country nm': ['C1', 'C2', 'C3', 'C4', 'C5', 'C6',
```

```
                    'C7', 'C8', 'C9', 'C10', 'C11', 'C12'],
        'child_mortality': [90.2, 16.6, 27.3, 119.0, 10.3, 14.5,
                            18.1, 4.8, 4.3, 39.2, 13.8, 8.6],
        'exports': [10.0, 28.0, 38.4, 62.3, 45.5, 18.9,
                    20.8, 19.8, 51.3, 54.3, 35.0, 69.5],
        'health': [7.58, 6.55, 4.17, 2.85, 6.03, 8.1,
                   4.4, 8.73, 11.0, 5.88, 7.89, 4.97],
        'imports': [44.9, 48.6, 31.4, 42.9, 58.9, 16.0,
                    45.3, 20.9, 47.8, 20.7, 43.7, 50.9],
        'income': [1610, 9930, 12900, 5900, 19100, 18700,
                   6700, 41400, 43200, 16000, 22900, 41100],
        'inflation': [9.44, 4.49, 16.1, 22.4, 1.44, 20.9,
                      7.77, 1.16, 0.87, 13.8, 0.39, 7.44],
        'life_expectancy': [56.2, 76.3, 76.5, 60.1, 76.8, 75.8,
                            73.3, 82.0, 80.5, 69.1, 73.8, 76.0],
        'total_fer': [5.82, 1.65, 2.89, 6.16, 2.13, 2.37,
                      1.69, 1.93, 1.44, 1.92, 1.86, 2.16],
        'gdp': [553, 4090, 4460, 3530, 12200, 10300,
                3220, 51900, 46900, 5840, 28000, 20700]
    })

features = df.select_dtypes(include=['float64', 'int64'])
scaler = StandardScaler()
X_scaled = scaler.fit_transform(features)

kmeans_3 = KMeans(n_clusters=3, random_state=42, n_init=10)
df['Cluster'] = kmeans_3.fit_predict(X_scaled)
print(df[['country nm', 'Cluster']].head())

wcss = []
k_range = range(1, 11)
for k in k_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X_scaled)
    wcss.append(km.inertia_)

plt.plot(k_range, wcss, marker='o')
plt.title("Elbow Method for Optimal k")
plt.xlabel("Number of Clusters (k)")
plt.ylabel("WCSS (Inertia)")
plt.show()
```

Slip 17

Q1) Write a program to implement the Best First Search Algorithm using heuristic values to find a path from a start node to a goal node. [10 marks]

```
import heapq

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': ['F'],
```

```
    'E': ['F'],
    'F': []
}

heuristics = {
    'A': 6, 'B': 5, 'C': 3, 'D': 2, 'E': 1, 'F': 0
}

def best_first_search(graph, h, start, goal):
    pq = [(h[start], start, [start])]
    visited = set()

    while pq:
        _, current, path = heapq.heappop(pq)

        if current == goal:
            return path

        if current not in visited:
            visited.add(current)
            for neighbor in graph[current]:
                if neighbor not in visited:
                    next_path = path + [neighbor]
                    heapq.heappush(pq, (h[neighbor], neighbor, next_path))

    return None

path = best_first_search(graph, heuristics, 'A', 'F')
print("Best First Search Path:", " -> ".join(path))
```

Q2) Write a program to implement Propositional Logic and evaluate logical expressions using operators such as AND, OR, and IMPLIES. [20 marks]

```
def implies(p, q):
    return (not p) or q

def evaluate_logic():
    print("P    Q       P AND Q P OR Q  P -> Q (IMPLIES)")
    print("-" * 50)
    for P in [True, False]:
        for Q in [True, False]:
            print(f"{P} {Q}     {P and Q}       {P or Q}        {implies(P, Q)}")

evaluate_logic()
```

OR Q2) Write a program to calculate and demonstrate the Euclidean Distance between two data points in Machine Learning. [20 marks]

```
import math

raw_p1 = input("Data Point 1 (e.g., 2 3 4): ") or "2 3 4"
raw_p2 = input("Data Point 2 (e.g., 5 7 6): ") or "5 7 6"

p1 = [int(x) for x in raw_p1.split()]
p2 = [int(x) for x in raw_p2.split()]
```

```
distance = math.sqrt(sum((a - b) ** 2 for a, b in zip(p1, p2)))

print("Data Point :", p1)
print("Data Point :", p2)
print(f"Euclidean Distance = {distance:.3f}")
```

Slip 21

Q1) Write a program implement Iterative Deepening Search (IDS). [10 marks]

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

def dls(node, goal, depth, path):
    if node == goal:
        return path
    if depth <= 0:
        return None
    for neighbor in graph[node]:
        res = dls(neighbor, goal, depth - 1, path + [neighbor])
        if res is not None:
            return res
    return None

def iterative_deepening_search(start, goal, max_depth):
    for depth in range(max_depth + 1):
        result = dls(start, goal, depth, [start])
        print(f"Searching at Depth Limit {depth}...")
        if result is not None:
            return result
    return None

path = iterative_deepening_search('A', 'E', max_depth=3)
print("IDS Path Found:", " -> ".join(path))
```

Q2) Write a program to implement the Gaussian Naive Bayes Classifier for classifying numerical data. [20 marks]

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

iris = load_iris()
X, y = iris.data, iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

```
model = GaussianNB()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
print("Sample Predictions:", y_pred[:10])
print("Actual Labels     :", y_test[:10])
print("Accuracy Score    :", round(accuracy_score(y_test, y_pred), 4))
```

OR Q2) Write a program to solve the Map Colouring Problem using the Constraint Satisfaction Problem (CSP) approach. [20 marks]

```
graph = {
    'A': ['B', 'C', 'E'],
    'B': ['A', 'C', 'D', 'E'],
    'C': ['A', 'B', 'E', 'F'],
    'D': ['B', 'E'],
    'E': ['A', 'B', 'C', 'D', 'F'],
    'F': ['C', 'E']
}

colors = ['Red', 'Green', 'Blue', 'Yellow']

def is_safe(node, color, assignment, graph):
    for neighbor in graph[node]:
        if assignment.get(neighbor) == color:
            return False
    return True

def solve_csp(nodes, graph, colors, assignment={}):
    if len(assignment) == len(nodes):
        return assignment

    unassigned = [n for n in nodes if n not in assignment][0]

    for color in colors:
        if is_safe(unassigned, color, assignment, graph):
            assignment[unassigned] = color
            result = solve_csp(nodes, graph, colors, assignment)
            if result:
                return result
            del assignment[unassigned]

    return None

solution = solve_csp(list(graph.keys()), graph, colors, {})
for node, color in solution.items():
    print(f"Node {node} -> {color}")
```

Slip 22

Q1) Write a program to solve the Tower of Hanoi Problem using state-space representation. [10 marks]

```
pegs = {'A': [3, 2, 1], 'B': [], 'C': []}

def tower_of_hanoi(n, source, auxiliary, target):
    if n == 1:
        disk = pegs[source].pop()
        pegs[target].append(disk)
        print(f"Move disk {disk} from {source} to {target} | {pegs}")
        return

    tower_of_hanoi(n - 1, source, target, auxiliary)
    disk = pegs[source].pop()
    pegs[target].append(disk)
    print(f"Move disk {disk} from {source} to {target} | {pegs}")
    tower_of_hanoi(n - 1, auxiliary, source, target)

print("Initial State:", pegs)
tower_of_hanoi(3, 'A', 'B', 'C')
```

Q2) Write a program to solve the N-Queens Problem using the Backtracking Algorithm. [20 marks]

```
N = 4
board = [[0] * N for _ in range(N)]

def is_safe(board, row, col):
    for i in range(row):
        if board[i][col] == 1:
            return False
    for i, j in zip(range(row - 1, -1, -1), range(col - 1, -1, -1)):
        if board[i][j] == 1:
            return False
    for i, j in zip(range(row - 1, -1, -1), range(col + 1, N)):
        if board[i][j] == 1:
            return False
    return True

def solve_n_queens(board, row=0):
    if row == N:
        return True

    for col in range(N):
        if is_safe(board, row, col):
            board[row][col] = 1
            if solve_n_queens(board, row + 1):
                return True
            board[row][col] = 0

    return False

solve_n_queens(board)
```

```
for r in board:
    print(r)
```

OR Q2) Write a program to design a Rule-Based Expert System for simple decision-making. [20 marks]

```
def loan_approval_expert_system(credit_score, income, has_default):
    if has_default:
        return "Decision: Loan Rejected (Default on record)."
    elif credit_score >= 750 and income >= 40000:
        return "Decision: Loan Approved (Prime Rate)."
    elif credit_score >= 650 and income >= 30000:
        return "Decision: Loan Approved (Standard Rate)."
    else:
        return "Decision: Loan Rejected (Low score or income)."

print("Applicant 1:", loan_approval_expert_system(780, 55000, False))
print("Applicant 2:", loan_approval_expert_system(680, 32000, False))
print("Applicant 3:", loan_approval_expert_system(720, 60000, True))
```

Slip 23

Q1) Write a program to demonstrate the Hill Climbing Algorithm for finding the maximum value of a simple objective function. (F(x) = -x^2 + 4x) [10 marks]

```
def F(x):
    return -(x ** 2) + 4 * x

def hill_climbing(start_x=0.0, step=0.1):
    current_x = start_x
    while True:
        neighbors = [round(current_x - step, 4), round(current_x + step, 4)]
        best_neighbor = max(neighbors, key=F)

        if F(best_neighbor) <= F(current_x):
            return current_x, F(current_x)

        current_x = best_neighbor

best_x, max_val = hill_climbing(start_x=0.0, step=0.1)
print(f"Optimal x = {best_x}")
print(f"Maximum Value F(x) = {max_val}")
```

Q2) Write a program to solve the Map Colouring Problem using the Constraint Satisfaction Problem (CSP) approach. [20 marks]

```
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'C', 'D'],
    'C': ['A', 'B', 'E'],
    'D': ['B', 'E'],
    'E': ['C', 'D']
}

colors = ['Red', 'Green', 'Blue']
```

```
def is_safe(node, color, assignment, graph):
    for neighbor in graph[node]:
        if assignment.get(neighbor) == color:
            return False
    return True

def solve_csp(nodes, graph, colors, assignment={}):
    if len(assignment) == len(nodes):
        return assignment

    unassigned = [n for n in nodes if n not in assignment][0]

    for color in colors:
        if is_safe(unassigned, color, assignment, graph):
            assignment[unassigned] = color
            result = solve_csp(nodes, graph, colors, assignment)
            if result:
                return result
            del assignment[unassigned]

    return None

solution = solve_csp(list(graph.keys()), graph, colors, {})
for node, color in solution.items():
    print(f"Node {node} -> {color}")
```

OR Q2) Write a program or report to compare Supervised Learning and Unsupervised Learning techniques. [20 marks]

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.cluster import KMeans
from sklearn.metrics import accuracy_score, silhouette_score

iris = load_iris()
X, y = iris.data, iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
sup_model = LogisticRegression(max_iter=200)
sup_model.fit(X_train, y_train)
sup_acc = accuracy_score(y_test, sup_model.predict(X_test))

unsup_model = KMeans(n_clusters=3, random_state=42, n_init=10)
cluster_labels = unsup_model.fit_predict(X)
unsup_score = silhouette_score(X, cluster_labels)

print("Supervised Accuracy     :", round(sup_acc, 4))
print("Unsupervised Silhouette :", round(unsup_score, 4))
```

Slip 25

Q1) Write the program to implement the AO* Algorithm [10 marks]

```
ao_graph = {
    'A': [[('B', 2)], [('C', 3)]],
    'B': [[('D', 1), ('E', 4)]],
    'C': [[('F', 2)], [('G', 5)]],
    'D': [[('H', 2)], [('I', 3)]],
    'E': [[('J', 1)]]
}

h = {node: 0 for node in 'ABCDEFGHIJ'}

def ao_star(node):
    if node not in ao_graph:
        return h[node], node

    best_cost = float('inf')
    best_path = None

    for branch in ao_graph[node]:
        branch_cost = 0
        sub_paths = []
        for child, edge_weight in branch:
            child_cost, child_path = ao_star(child)
            branch_cost += edge_weight + child_cost
            sub_paths.append(child_path)

        if branch_cost < best_cost:
            best_cost = branch_cost
            best_path = f"{node} -> ({' AND '.join(sub_paths)})"

    h[node] = best_cost
    return best_cost, best_path

min_cost, solution_graph = ao_star('A')
print("AO* Minimum Cost:", min_cost)
print("Optimal Subgraph:", solution_graph)
```

Q2) Write a program to implement the Minimax Algorithm for a two-player game. [20 marks]

```
game_tree = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': 3,
    'E': 5,
    'F': 2,
    'G': 9
}

def minimax(node, is_max):
    if isinstance(game_tree[node], int):
        return game_tree[node]
```

```
    scores = [minimax(child, not is_max) for child in game_tree[node]]
    return max(scores) if is_max else min(scores)

print("Value at MIN Node B:", minimax('B', False))
print("Value at MIN Node C:", minimax('C', False))
print("Optimal Value at Root A (MAX):", minimax('A', True))
```

OR Q2) Write a program to implement the Forward Chaining inference mechanism. [20 marks]

```
def forward_chaining(facts, rules, goal):
    inferred = set(facts)
    changed = True

    while changed:
        changed = False
        for conditions, result in rules:
            if conditions.issubset(inferred) and (result not in inferred):
                inferred.add(result)
                print(f"Inferred '{result}' from {conditions}")
                new_fact_added = True
                if result == goal:
                    return True

    return goal in inferred

facts = {"rains", "cloudy"}
rules = [
    ({"rains"}, "ground_is_wet"),
    ({"ground_is_wet", "cloudy"}, "carry_umbrella")
]
goal = "carry_umbrella"

is_proved = forward_chaining(facts, rules, goal)
print(f"Goal '{goal}' proved:", is_proved)
```